JFVP
JUSTICE FOR FRAUD VICTIMS PROJECT

IMPLEMENTATION GUIDE

Bank Statement Analyzer

Hosting the analyzer securely at your university or organization.

For academic leadership, faculty and IT departments

Version 1.1 · 2026

Contents

01	Summary for decision makers	3
02	Two ways to host it	4
03	Running a pilot	5
04	Option 1: Browser version	6
05	Option 2: Server version	7
06	How the analyzer protects data	9
07	Who is responsible for what	10
08	Approvals, updates and support	11
09	IT checklist	12

1. Summary for decision makers

Bank Statement Analyzer gives forensic accounting students and fraud investigators a professional tool for reading bank statements, reconciling them, spotting fraud patterns and producing workpapers. It is free, it is open source, and it can run on your university's own website.

What it does

Reads bank and credit card statements from any bank, checks every account against the statement's own balances, flags patterns such as structuring and a draining balance, checks files for signs of editing, and exports Excel workpapers.

Who it's for

Forensic accounting and fraud examination courses, student clinics working real cases under supervision, and investigators supporting fraud victims.

What it costs

Nothing. The software and its updates are free under the Apache License 2.0. Your only costs are a small web server and staff time, plus an independent security review if your policies require one.

How data is protected

In the recommended setup, statements are read inside each person's browser and are never sent to the university's server or anyone else. Case projects are encrypted with a password.

RECOMMENDATION

Start with **Option 1, the browser version**, for one course or clinic. The university's server holds no case data, which keeps the security review simple. Add Option 2 later only if users need features it provides.

What we ask of a pilot university

- A faculty sponsor with a course or clinic that will use it for one term.
- An IT contact to host the browser version and handle the university's usual security review.
- Feedback at the end of the term on what worked and what's missing.

2. Two ways to host it

	Option 1: Browser version (recommended)	Option 2: Server version
Where statements are read	In each person's web browser. Never uploaded.	On your server. Uploaded to it.
Case data on your server	None	Yes, in encrypted projects
What the server does	Delivers the app's files, like any static website	Runs the app
Sign-in	Optional (the server holds no data)	Required, using your university login
Scanned paper statements	Not yet (users run the computer version for scans)	Yes
Saved projects	Encrypted, in each person's browser; downloadable	Encrypted, on the server, separate per person
Server needed	Any web server; about 85 MB of files	2 CPUs, 4 GB memory, and disk for case data
Approval effort	Low: a website that stores nothing	Higher: a system that stores case data

Both options come with ready-made Docker container images and step-by-step instructions. A third way needs no hosting at all: each investigator runs the analyzer on their own computer, which also reads scanned statements.

3. Running a pilot

A typical pilot takes about one academic term.

When	Steps	Who
Weeks 1–2	Agree on the pilot course. IT reviews this guide and the security documentation, and completes any internal security questionnaire (such as HECVAT).	Faculty sponsor, IT
Week 3	IT deploys Option 1 on a university web address and runs the checklist in section 9.	IT
Week 4	Faculty try it with the built-in demo case and plan the exercises (see the User Guide).	Faculty
Weeks 5–14	Students use it in class with the demo case. Clinics may use it on real cases under the handling rules in section 7.	Faculty, students
Week 15	Review: usefulness, statements that didn't read correctly, feature requests, any security questions.	All, with JFVP

Measures of success

- [ ] Students complete the demo-case exercises and produce workpapers.
- [ ] Real statements used in clinics reconcile, or the variances are understood.
- [ ] No case data leaves approved systems.
- [ ] Faculty want to keep using it in the next term.

4. Option 1: Browser version

Your server hands out the app's files. Each person's browser runs the app and reads statements on their own computer.

The page carries a strict security policy: the browser may load from, and connect to, your server only. That applies to the code that reads statements too, so statements cannot be sent anywhere, even by a software bug or a tampered component. In testing, the complete app ran with every outside connection blocked, and a deliberate attempt to contact another website was refused by the browser.

With Docker

```
docker run -d --name bank-statement-analyzer -p 8080:8080 --read-only --tmpfs /tmp \
  --restart unless-stopped ghcr.io/jdaiken/bank-statement-analyzer-browser:<version>
```

Images are published with each release. To build the image yourself from the source code:

```
git clone https://github.com/jdaiken/parsing_tool.git && cd parsing_tool
docker build -f deploy/browser/Dockerfile -t bank-statement-analyzer-browser .
```

The container runs the nginx web server as an unprivileged user on a read-only filesystem, accepts only read requests (an upload attempt returns 403), and sends the security headers. Put it behind your HTTPS load balancer or reverse proxy. On a network that inspects HTTPS traffic, pass your organization's certificate to the build: `docker build --secret id=ca_bundle,src=ca.pem ...` (used during the build only, never stored in the image).

On an existing web server, without Docker

1. Download **bank-statement-analyzer-offline-<version>.zip** from the project's GitHub Releases page, or build it with `python tools/build_offline.py`.
2. Check the zip against its published **.sha256** fingerprint, and the unzipped folder against the **SHA256SUMS.txt** file inside it.
3. Copy the folder to a site served over HTTPS.
4. Turn on the security headers with the included settings: **.htaccess** (Apache, needs mod_headers), **web.config** (IIS) or **nginx.conf.example**. They also set the right file types (for example, `.wasm` as `application/wasm`).

WHAT USERS EXPERIENCE

The first visit loads about 85 MB from your server, which is then cached. Projects are saved encrypted in that browser; users should download projects they need to keep, because clearing browser data erases them. For scanned statements, users run the computer version.

5. Option 2: Server version

The full app, including reading scanned statements, runs on your server behind your university login. Use it only on a server your institution approves for case data.

What's included

Sign-in container

oauth2-proxy sends people to your identity provider (Microsoft Entra ID, Google Workspace, Okta or any OpenID Connect provider) and tells the app who signed in.

App container

On an internal network with no published port and no internet access, so it can only be reached through sign-in. Unprivileged, read-only, all Linux capabilities dropped. Case data lives in a dedicated volume.

Setup

1. **Register the app with your identity provider** as an OpenID Connect web application, with the redirect address `https://<your-host>/oauth2/callback`. Limit who can sign in at the provider.
2. **Configure**: copy `deploy/server/.env.example` to `.env` and fill in the issuer address, client ID, client secret, cookie secret, allowed email domains and public address.
3. **Start**:

```
docker compose -f deploy/server/docker-compose.yml --env-file deploy/server/.env up -d --build
```

4. **Put it behind HTTPS**: point your reverse proxy at `127.0.0.1:4180`. It must pass WebSocket connections through.
5. **Pin the sign-in image** to an exact version (a digest) in `docker-compose.yml`.
6. **Protect the data**: keep the case-data volume on an encrypted disk, include it in backups, and limit who can log in to the server.

How it keeps people's cases apart

Two settings, already in `docker-compose.yml`, turn on server mode. Set them only on an approved server:

- `BSA_APPROVED_SERVER=1` removes the "demonstration copy" warning.
- `BSA_USER_HEADER=X-Forwarded-Email` names the sign-in information the app trusts. The app keeps a separate project folder for each person and refuses to run without sign-in. Never expose the app container directly: that information is trusted only because sign-in is the only way in.

SERVER ADMINISTRATORS

In this option statements are on your server while they're analyzed, and projects are stored there (encrypted with their owners' passwords). People with administrative access to that server are inside the trust boundary.

6. How the analyzer protects data

Protection	How it works
Statements stay local	Option 1 and the computer version read statements on the user's computer. The computer version only accepts connections from that computer and sends no usage statistics.
No outside connections	The browser version's security policy allows connections to its own server only. Fonts and chart code are built in, so no outside service is ever called.
Encrypted projects	Statements, their file names, notes and the case log are encrypted with AES-256-GCM, with a key derived from the project password using scrypt (an OWASP-recommended setting). Tampering is detected. Lost passwords can't be recovered.
Encrypted sharing	Projects are shared as encrypted .bsacase files with their own password.
Evidence integrity	Each statement's SHA-256 fingerprint, the app version, the analyst and the time are recorded in every workpaper. Every transaction names its source file, page and line. Original files are never changed.
Leaky redactions	Text hidden under black boxes but still in the PDF is detected, hidden, and reported.
Labelled demo copies	A copy running on a server that isn't approved shows a red warning not to upload real statements.
Supply chain	Every software component is pinned to an exact version and verified by its fingerprint. Container base images and build automation are pinned to exact digests. Automated tests run on every change, and security updates are proposed weekly.

Not covered by the software: workpapers and exports are ordinary Excel, CSV and HTML files (not encrypted); a compromised computer or browser extension can see what's on screen; and a clean document check doesn't prove a statement is genuine.

7. Who is responsible for what

Area	University IT	Faculty and supervisors	Users
Hosting	Deploy, HTTPS, security headers, updates	—	—
Access	Sign-in and access control (Option 2)	Decide who works on real cases	Keep passwords private
Devices	Encryption and update policies for university devices	—	Encrypted device, screen lock, no shared computers
Case files	Approved storage locations; backups (Option 2)	Retention schedule; authorization from victims	Store exports only in approved storage; use case numbers, not names
Students on real cases	—	Supervision; confidentiality agreements	Follow the handling rules in the User Guide
Incidents	Lead the response	Report immediately	Report immediately
Software releases	Published by the project maintainers, with fingerprints, on the project's GitHub page.		

LEGAL REVIEW

Have your institution's counsel review the data-handling policy for real cases, especially where findings may go to law enforcement or court. State breach-notification laws can apply when names together with account numbers are exposed.

8. Approvals, updates and support

Approvals universities usually need

Item	Status
License	Apache License 2.0: free to use, copy, modify and host, permanently. © 2026 Justice for Fraud Victims Project.
Security questionnaire (HECVAT)	Answers can be prepared from the project's security documentation. For Option 1, many items don't apply because no institutional data is stored.
Independent security review	The code is open source and available for review. Arrange an independent test if your policies require one.
Accessibility (VPAT)	The app is a standard web page. A formal accessibility review has not yet been completed; ask if your institution requires one.
Privacy	Option 1 collects no personal data. Option 2 stores the signed-in email address (as a folder fingerprint) and users' encrypted projects.

Updates

New versions are published as releases on the project's GitHub page, each with the offline package, its fingerprint, and both container images. To update, pull the new image (or replace the folder) and restart; projects and case data carry over. Watch the repository's releases to be notified.

Support and security reports

Questions and feature requests: through the project's GitHub page or justiceforfraudvictimsproject.org. Security problems: report privately through **Report a vulnerability** on the repository's Security tab, not in a public issue.

Reference documents in the repository

- **docs/UNIVERSITY_DEPLOYMENT.md**: the technical deployment guide.
- **docs/SECURITY_AND_DATA_HANDLING.md**: the full security and data handling guide.
- **SECURITY.md**: the security policy and how to report problems.
- **User Guide**: for students, instructors and investigators.

9. IT checklist

Before go-live (Option 1)

- [ ] Files match **SHA256SUMS.txt** (or the image was built from a tagged release).
- [ ] Served over HTTPS on a university address.
- [ ] `curl -sI https://<host>/index.html` shows **Content-Security-Policy** with `connect-src 'self' blob: data:`, plus X-Content-Type-Options, Referrer-Policy and Permissions-Policy.
- [ ] `curl -X POST https://<host>/index.html` returns 403.
- [ ] In the browser's developer tools (Network tab), loading the sample statements and downloading a workpaper shows requests to your host only.
- [ ] Typing `fetch("https://example.com")` in the browser console is refused with a Content Security Policy error.
- [ ] The header reads "Private: runs inside your browser" and there is no red demonstration warning.

Before go-live (Option 2)

- [ ] Visiting the site without signing in redirects to your identity provider.
- [ ] The app container has no published port; only the sign-in container is reachable, through HTTPS.
- [ ] Two different people do not see each other's projects.
- [ ] Reading the scanned demo statement works, and it reconciles.
- [ ] The case-data volume is on an encrypted disk and in the backup schedule.
- [ ] The sign-in image is pinned to a digest; `.env` is kept out of version control.

Ongoing

- [ ] Someone is assigned to watch for new releases and apply them.
- [ ] Access to the server (Option 2) is reviewed each term.
- [ ] Incidents are reported and handled under your institution's process.